

A.1 - Navegación Robótica Autónoma con Arucos

Link:

<https://randomnerdtutorials.com/esp32-cam-video-streaming-face-recognition-arduino-ide/>

El objetivo del proyecto es construir un robot móvil que se ubica y navega de forma autónoma dentro de un espacio definido utilizando marcadores visuales (Arucos) como puntos de referencia.

- **El Hardware:** Un ESP32-CAM (que incluye el microcontrolador y la cámara), un chasis de robot con dos motores y ruedas, y un controlador de motor (como el L298N).
- **El Desafío de Tiempo Real:** El ESP32 debe capturar imágenes, procesarlas para detectar Arucos, calcular su posición y orientación actual, y ajustar la velocidad de los motores para seguir una ruta, repitiendo este ciclo completo en una fracción de segundo para garantizar una navegación fluida y precisa. Para esto se deben definir 3 task solamente, que cumplan con lo siguiente:
 - **Tarea 1 (Baja Prioridad): Procesamiento de Imagen**, que captura y analiza el video para localizar los Arucos.
 - **Tarea 2 (Media Prioridad): Planificación de Ruta**, que decide el siguiente movimiento basándose en la posición actual.
 - **Tarea 3 (Alta Prioridad): Control de Motores**, que ejecuta el movimiento de forma precisa.

A2 - Robot de Equilibrio Automático (Péndulo Invertido)

Link: <https://www.instructables.com/Stabilize-Sensor-Readings-With-Kalman-Filter/>

El objetivo del proyecto es construir un pequeño robot de dos ruedas que se equilibra a sí mismo, como un Segway.

- **El Hardware:** Un ESP32, un sensor de giroscopio y acelerómetro (como el MPU6050), drivers de motor y dos motores.
- **El Desafío de Tiempo Real:** El ESP32 debe leer los datos del sensor, calcular el ángulo de inclinación y ajustar la velocidad de los motores cientos de veces por segundo.
 - **Tarea 1 (Prioridad Crítica): Bucle de Equilibrio.** Esta tarea debe ejecutarse con una frecuencia y regularidad perfectas (ej., cada 4 milisegundos) para leer el acelerómetro y calcular el ángulo de inclinación mediante el **filtro de kalman**.
 - **Tarea 2 (Baja Prioridad): Comunicación.** Podría enviar datos de telemetría a un server, visualizarlos, y además, tener un control remoto con 2 botones (adelante y atrás).

A3 - Sistema de Tracking de Activos con Alerta de "Hombre Caído"

El objetivo es crear un dispositivo portátil que envía su ubicación GPS a una central a intervalos regulares. Constantemente se monitorea con un acelerómetro que el usuario se encuentre "de pie". Si se detecta un patrón de caída brusca seguido de inmovilidad, se debe enviar una alerta de emergencia con máxima prioridad.

- **El Hardware:** Un TTGO LoRa, un módulo GPS (tipo arduino), un acelerómetro (MPU6050) y una batería. Otro TTGO LoRa como receptor.
- **El Desafío de Tiempo Real:** Tienes dos tareas principales: el reporte periódico de GPS (baja prioridad) y el monitoreo de caídas (máxima prioridad). El sistema debe garantizar que, si se detecta una caída, la tarea de enviar la alerta de emergencia **interrumpa inmediatamente** a cualquier otra tarea (como el envío normal de GPS) para que la señal de auxilio se envíe en menos de, por ejemplo, 1 segundo. Es un problema clásico de prioridades y tareas críticas en un sistema de tiempo real.
 - **Tarea 1 (Prioridad Crítica): Monitoreo de Acelerómetro.** Se ejecuta constantemente a alta frecuencia (ej. 100 Hz), analizando los datos del MPU6050 para detectar el patrón de una caída. Si se confirma una emergencia, debe disparar la alerta inmediatamente.
 - **Tarea 2 (Baja Prioridad): Reporte Periódico de GPS.** Se activa a intervalos largos (ej. cada 30 segundos) para enviar la ubicación de rutina a través de LoRa.
 - **Tarea 3 (Baja Prioridad): Procesamiento de Datos GPS.** Una tarea de fondo que continuamente lee y procesa el flujo de datos del módulo GPS para tener la última coordenada válida siempre lista.

A4 - Sistema de Riego - Control Remoto de Actuador con Mecanismo "Fail-Safe" (A prueba de fallos)

El objetivo es crear un sistema para controlar a distancia algo crítico, como una válvula de riego o el encendido de una bomba. Un nodo remoto (el actuador) solo se mantendrá activo si recibe una señal de "heartbeat" o "estoy vivo" desde la base de control cada X segundos.

- **El Hardware:** 1 ESP32 y un servidor MQTT. Una es la base con un botón/interruptor. La otra está conectada a un relé o un servo para controlar la válvula/bomba. La intención es tenerlo al ESP32 en bajo consumo y configurarlo para que escuche X inputs de señales.
- **El Desafío de Tiempo Real:** El reto es garantizar la seguridad del sistema mediante un mecanismo de "watchdog" que debe funcionar de manera infalible, incluso con el ESP32 operando en modos de bajo consumo. Esto se gestiona con dos tareas principales en el RTOS:
 - **Tarea 1 (Prioridad Crítica): Watchdog de Conexión.** Un temporizador de alta prioridad que se reinicia con cada señal de "heartbeat" válida recibida vía MQTT. Si este temporizador expira (ej. tras 10 segundos sin señal), la tarea

toma control inmediato del relé para llevarlo al estado seguro (ej. cerrar la válvula).

- **Tarea 2 (Media Prioridad): Cliente MQTT.** Se encarga de conectarse a la red, escuchar los mensajes del servidor, ejecutar los comandos recibidos ("ABRIR", "CERRAR") y, fundamentalmente, reiniciar el temporizador del watchdog al recibir un "heartbeat".

El desafío central es que el RTOS debe asegurar que la **Tarea Watchdog** sea soberana. Tiene que poder ejecutarse y activar el modo seguro de forma determinista, sin importar si la Tarea MQTT está ocupada, bloqueada intentando reconectar al Wi-Fi, o si el procesador acaba de despertar de un ciclo de sueño profundo.

A5. Análisis de Cómputo y Comunicación entre Arduino DUE y Mega

El objetivo es medir y comparar la latencia de cómputo y comunicación entre un microcontrolador de 8-bits (Arduino Mega) y uno de 32-bits de alta velocidad (Arduino DUE), para determinar los cuellos de botella en una tarea de procesamiento distribuido. El objetivo final es *multiplicar dos matrices de 30x30 ($C=A \times B$) requiere **27,000 multiplicaciones** y una cantidad similar de sumas.*

El Hardware: 1 Arduino DUE, 1 Arduino Mega, y cables para conectar sus puertos serie (TX con RX y RX con TX).

El Desafío de Tiempo Real: El reto es cuantificar y descomponer la latencia total de una tarea distribuida (multiplicación de matrices) para identificar si el sistema está limitado por el cómputo o por la comunicación. Esto requiere una medición precisa del tiempo gestionada por tres "fases" o "tareas" lógicas:

- **Tarea 1** (Orquestador y Medición): Reside en el Arduino DUE (actuando como maestro). Es responsable de particionar la matriz, iniciar un temporizador, delegar parte del trabajo al Mega (parte de la matriz para calcular, se encuentra en el DUE) y medir el tiempo total de la operación de ida y vuelta.
- **Tarea 2** (Comunicación Serial): Gestiona la transmisión y recepción de los datos (fragmentos de la matriz y resultados) a través de los pines TX/RX, asegurando la integridad de los datos.
- **Tarea 3** (Cómputo Esclavo): Se ejecuta en el Arduino Mega. Espera recibir los datos, realiza su porción de la multiplicación de la matriz y devuelve el resultado.

El desafío principal es usar los temporizadores de hardware e interrupciones para medir con exactitud la duración de cada fase (serialización, transmisión, cómputo remoto y recepción).